

Docker Compose Architecture Guide

Laravel, Nginx, Node.js (Vite/Vue), MySQL, Redis & Mailpit Stack

Project Location: C:\Users\MSI\Documents\CODE\side-project\my-app

Docker Configuration: C:\Users\MSI\Documents\CODE\side-project\docker

Author: Antigravity AI Coding Assistant

Date: June 2026

1. Architectural Overview

This containerized development setup uses a multi-container Docker Compose structure. To keep the Laravel codebase clean, the configuration files are located in a sibling `docker` directory.

By utilizing host volumes, the project files are bind-mounted directly into the container. This ensures that any edit made in your editor is reflected instantly inside the running containers, enabling standard web development practices with zero compilation lag.

Note: All dependencies (PHP, Composer, Node, NPM, MySQL, Redis) run exclusively inside Docker. No runtime is required on the host system, keeping your computer clean.

Directory Tree

```
C:\Users\MSI\Documents\CODE\side-project\  
├─ docker/  
|   ├─ docker-compose.yml      # Service declarations  
|   ├─ Dockerfile              # PHP FPM image compilation instructions  
|   ├─ docker-entrypoint.sh    # Container bootstrapper and dependency installer  
|   └─ nginx.conf              # Nginx server virtual host definition  
└─ my-app/                    # Your Laravel Application (Bind-mounted)  
    ├─ package.json  
    ├─ vite.config.js          # Modified for Hot Module Replacement (HMR)  
    └─ ...
```

2. Container Services Breakdown

The setup initializes 6 distinct containers, communicating via a private bridge network:

1. PHP FPM (`php`)

Runs the Laravel core backend. It uses PHP 8.3 FPM to execute controllers, handle database queries, run Artisan commands, and process background logic.

2. Nginx Web Server (`nginx`)

Handles incoming web requests on port `8000` . It serves static assets directly and proxies PHP executions to the PHP FPM container on port `9000` .

3. MySQL Database (`db`)

Acts as the relational database on port `3306` . Stores persistent application data using a Docker named volume (`db_data`) for data preservation.

4. Redis Cache (`redis`)

Handles session state, application cache, and high-speed queue operations on port `6379` . Data is persisted to a named volume (`redis_data`).

5. Node.js Engine (`node`)

Runs Node.js 20. It executes `npm install` , builds client assets, and runs the Vite hot-reloading dev server exposed on port `5173` .

6. Mailpit SMTP (`mailpit`)

Intercepts all outgoing emails sent by the Laravel app (SMTP on port `1025`). Provides a gorgeous Web UI on port `8025` to read intercepted emails.

3. Ports Mapping Reference

Service	Host Port	Container Port	Protocol	Purpose
Nginx	8000	80	HTTP	Web Access to Laravel App
Node/Vite	5173	5173	TCP	Vite Asset HMR / CSS & JS reload
MySQL	3306	3306	TCP	DB Connection (e.g. TablePlus, DBeaver)
Redis	6379	6379	TCP	Redis CLI access / Cache monitoring
Mailpit Web	8025	8025	HTTP	Web interface to view sent emails

4. Inside the Configuration Files

A. Dockerfile (PHP FPM)

Based on `php:8.3-fpm`, it installs system utilities (`git`, `curl`, `zip`), compiles required database and image extensions (`pdo_mysql`, `mbstring`, `gd`, `zip`), installs `redis` and `xdebug` PECL extensions, pulls down the official Composer binary, and sets up a default non-root user (`developer`).

B. docker-entrypoint.sh

Executed every time the `php` container boots. It does the following:

1. Copies `.env.example` to `.env` if missing.
2. Modifies database, mail, and cache hostnames in `.env` automatically so Laravel reaches the Docker services (e.g. changing `DB_HOST=127.0.0.1` to `DB_HOST=db`).
3. Checks if the `vendor/` directory exists; if not, runs `composer install`.
4. Generates an encryption key via `php artisan key:generate` if not already set.
5. Fixes permissions for `storage` and `bootstrap/cache` to prevent webserver write errors.
6. Delegates process execution to `php-fpm`.

C. nginx.conf

Serves the public folder (`/var/www/html/public`). Utilizes `fastcgi_pass php:9000;` to forward requests ending in `.php` to the FPM service inside the bridge network.

D. vite.config.js (HMR Hot-fix)

When Vite runs inside Docker, it binds to `0.0.0.0` to listen to incoming requests. By default, it publishes its local container loopback address (such as `[::]` or `172.20.x.x`) to `public/hot`. Since your host browser cannot resolve the internal container IP, we override HMR settings to announce `localhost`:

```
server: {  
  host: '0.0.0.0',  
  hmr: {  
    host: 'localhost',  
  },  
}
```

This writes `http://localhost:5173` to the `public/hot` file, ensuring the host browser successfully retrieves Javascript and CSS files.

5. Development Workflow Commands Cheatsheet

Since you do not have native binaries on your machine, you must run all CLI tools via Docker Compose. Ensure you are in the `docker` folder (`C:\Users\MSI\Documents\CODE\side-project\docker`) when running these:

Stack Management

- **Start all services:** `docker compose up -d`
- **Rebuild and start:** `docker compose up -d --build`
- **Stop all services:** `docker compose down`
- **Stop and wipe volumes:** `docker compose down -v`

Running Artisan Commands

Execute Laravel Artisan commands inside the running `php` container:

```
docker compose exec php php artisan migrate
docker compose exec php php artisan db:seed
docker compose exec php php artisan make:controller PostController
docker compose exec php php artisan route:list
```

Running Composer Commands

Pull down or update packages without host Composer:

```
docker compose exec php composer require laravel/sanctum
docker compose exec php composer update
```

Running Node/NPM Commands

Manage packages or build files inside the `node` container:

```
docker compose exec node npm install axios
docker compose exec node npm run build
```

Viewing Logs

Monitor output from specific containers:

```
docker compose logs -f php  
docker compose logs -f nginx  
docker compose logs -f node
```